Enterprise RAG Platform on Azure (AI Search + Private Data)

Implementation Solution Guide

Version 1.0.0 · 2026-07-10

Contents

Executive Summary — Enterprise RAG Platform on Azure (AI Search + Private Data) 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Outcome . . . 6

1. Solution Overview — Enterprise RAG Platform on Azure (AI Search + Private Data) 6

What the RAG platform is . . . 6

Why this matters . . . 7

How it works . . . 7

What you can control . . . 7

Limitations (be honest with buyers) . . . 8

Auth and connections used . . . 8

The standard deployment model . . . 9

2. Requirements Guide — Enterprise RAG Platform on Azure (AI Search + Private Data) 9

2.1 Azure AI Search service at a tier that supports semantic ranking + vector . . . 9

2.2 Azure OpenAI resource with an embedding deployment and a chat deployment . . . 9

2.3 A source data set with a known, resolvable permission (ACL) model . . . 10

2.4 Entra groups for the pilot content and test users . . . 10

2.5 Managed identity with least-privilege data-plane roles . . . 10

2.6 Python 3.x with the RAG SDK toolchain . . . 10

2.7 Private networking for the AI services . . . 11

2.8 Cost controls, quotas, and a non-prod environment with rollback . . . 11

Checklist . . . 11

Global Requirements — AI (category-level) 11

G1. Azure subscription with AI service access and rights . . . 12

G2. Entra identity, managed identity, and RBAC . . . 12

G3. Private networking capability . . . 12

G4. IaC + Python tooling and source control . . . 12

G5. CI/CD platform . . . 12

G6. Data source and classification (for data-grounded solutions) . . . 13

G7. Content-safety and responsible-AI posture . . . 13

G8. Cost controls and quotas . . . 13

3. Implementation Guide — Enterprise RAG Platform on Azure (AI Search + Private Data) 13

Naming (set once, reuse verbatim) . . . 13

Step 1 — Assess and set variables . . . 14

Step 2 — Deploy the infrastructure (Bicep what-if = the gate) . . . 14
Step 3 — Create the permission-trimmed index . . . 14
Step 4 — Ingest a small pilot set (dry-run first) . . . 15
Step 5 — Verify hybrid + semantic retrieval works . . . 15
Step 6 — Run the PERMISSION-TRIMMING negative test (the hard gate) . . . 15
Step 7 — Wire the grounded orchestration (prompt flow) . . . 16
Step 8 — Evaluate groundedness and relevance . . . 16
Step 9 — Turn on monitoring . . . 16
Step 10 — Expand to more content and production . . . 16
5. Testing Guide — Enterprise RAG Platform on Azure (AI Search + Private Data) 17
Test 1 — Infra is private and keyless . . . 17
Test 2 — The index has the security field . . . 17
Test 3 — Every chunk carries an ACL . . . 17
Test 4 — PERMISSION-TRIMMING negative test (the hard gate) . . . 17
Test 5 — Hybrid + semantic ranking are actually applied . . . 18
Test 6 — Grounded answer with citations . . . 18
Test 7 — Refusal when unsupported . . . 18
Test 8 — Groundedness / relevance evaluation . . . 18
Test 9 — Posture report . . . 18
Pass/fail gate . . . 18
6. Troubleshooting Guide — Enterprise RAG Platform on Azure (AI Search + Private Data) 18
A. The permission-trimming test FAILS (a restricted chunk reached an unauthorised user) . . . 19
B. Bicep deploy / infra failures . . . 19
C. Retrieval returns nothing or irrelevant chunks . . . 20
D. Answers are wrong, uncited, or hallucinated . . . 20
E. Auth / keyless issues . . . 21
F. Cost / throttling . . . 21
G. Diagnostics quick reference . . . 21
H. When to escalate . . . 21
7. Best Practices — Enterprise RAG Platform on Azure (AI Search + Private Data) 22
Where to go next . . . 23
8. Deliverables Summary — Enterprise RAG Platform on Azure (AI Search + Private Data) 23
Documentation . . . 23
Automation & scripts . . . 24
What the buyer receives . . . 24
Outcome . . . 24

Appendix A — PowerShell Scripts 26

Executive Summary — Enterprise RAG Platform on Azure (AI Search + Private Data)

White-label · AI

A chatbot that can't see your data hallucinates and is worthless. A chatbot that *can* see your data but ignores who is allowed to read it is worse — it is a **data-leak engine** that will happily quote a layoff memo or a salary spreadsheet to anyone who asks. This solution builds an **enterprise Retrieval-Augmented Generation (RAG) platform on Azure** that grounds Azure OpenAI on your own private content **and enforces the source's permissions on every answer** — so it responds from your data, cites where each fact came from, and **never returns content the asking user isn't allowed to see**.

The platform is delivered **as code**: an ingestion pipeline that chunks and embeds your documents, an **Azure AI Search** index with **hybrid (keyword + vector) search and semantic ranking**, a **permission-trimmed / security-filtered** retrieval layer that respects each document's ACLs, and an orchestration layer (grounded prompts, citations, evaluation) built with **prompt flow**. It is written for an administrator with no prior RAG, Azure AI Search, or prompt-engineering experience. Everything runs **private-by-default** (no public endpoints, keyless auth) and is proven in a **non-production** environment before it touches anything real.

What you get

- A plain-language **Solution Overview** of chunking, embeddings, hybrid + semantic + vector search, permission-trimming, and grounded orchestration.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite.
- An **Implementation Guide** of precise, ordered steps — deploy the infra (Bicep what-if first) → build the permission-trimmed index → run the ingestion/embedding pipeline → wire the grounded prompt flow → evaluate → expand — non-prod first.
- Real automation: a **Bicep** infrastructure template, and four **Python** scripts (`azure-search-documents`, `openai`, `promptflow`) for discovery, ingestion + embedding, permission-trimming validation, and reporting.
- **Testing** and **Troubleshooting** guides — including the critical **negative** permission-trimming test (a user must be *denied* content they can't see).

What this delivers

A working RAG platform: private documents **chunked and embedded** into an **Azure AI Search** index that carries a per-document **ACL field (**`group_ids`**)**; **hybrid retrieval** (BM25 keyword + vector similarity) re-ranked by **semantic ranking**; a **security filter applied on every query** so the index only ever returns chunks the caller's Entra groups are permitted to read; and a **grounded orchestration flow** that feeds only those permitted chunks to Azure OpenAI, returns an answer **with citations**, and is scored for **groundedness and relevance** by an evaluation harness — all validated in non-prod first.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with a **Bicep what-if gate** before any infra apply, a **dry-run gate** before any index write, and the permission-trimming **negative test** as a hard pass/fail gate before expansion.

Outcome

AI that answers from *your* data, shows its sources, and honors your permissions — instead of a chatbot that either makes things up or leaks. Deployable by a single administrator, proven in non-production, and reusable on every content set you onboard. **Data-classification, acceptable-AI-use, and records/regulatory determinations remain the customer's compliance, legal, and security decisions** — this guide provides the technical controls, not those decisions.

1. Solution Overview — Enterprise RAG Platform on Azure (AI Search + Private Data)

What the RAG platform is

Retrieval-Augmented Generation (RAG) makes a large language model answer from **your** content instead of only from what it was trained on. Rather than asking the model to "know" your policies, contracts, or tickets, you **retrieve** the most relevant passages from your own data at query time and hand them to the model as grounding, so the answer is based on real source text and can be **cited**. This solution builds that whole pipeline on Azure, and adds the one control most home-grown RAG demos skip: it **enforces the source document's permissions on every retrieval**.

It combines five capabilities:

- **Ingestion, chunking, and embedding (as code)** — a pipeline reads your source documents, splits them into overlapping **chunks** small enough to retrieve precisely, generates a **vector embedding** for each chunk with an Azure OpenAI embedding model, and writes the chunk text + vector + metadata into the search index.
- **Azure AI Search index — hybrid + semantic + vector** — each query runs **keyword (BM25)** and **vector (nearest-neighbour)** search together (**hybrid**), and the combined results are re-ordered by Azure AI Search **semantic ranking** so the passages the model sees are the most relevant, not just the most keyword-similar.
- **Permission-trimmed / security-filtered retrieval** — every chunk carries an **ACL field** (`group_ids`) copied from the source document's permissions. Every query is executed with a **security filter** that keeps only the chunks whose `group_ids` intersect the **asking user's Entra group memberships**. Content the user can't open in the source system is never even retrieved — so it can never reach the model or the answer.
- **Grounded orchestration with citations (prompt flow)** — a **prompt flow** takes the user's question, retrieves the permitted chunks, builds a **grounded prompt** ("answer only from the sources below; if the sources don't contain the answer, say so"), calls the Azure OpenAI chat model, and returns the answer **with citations** back to the specific source chunks.
- **Evaluation harness + monitoring** — an evaluation flow scores answers for **groundedness** (is the answer supported by the retrieved sources?) and **relevance** (does it address the question?), and the

platform's telemetry (Application Insights / Azure Monitor) tracks latency, token cost, and failure rates.

Why this matters

Two failure modes make private-data chatbots dangerous. The first is **hallucination**: an ungrounded model invents plausible-sounding answers, and users can't tell the fabricated ones from the real ones. RAG fixes this by grounding the answer in retrieved source text and citing it, so answers are checkable. The second — and the reason this solution exists — is the **data leak**: a naive RAG index ingests every document into one pool and returns the best semantic match **regardless of who is asking**. A contractor asks an innocent question and the model quotes a passage from a board deck or an HR file they were never allowed to open. The permission-trimming layer closes this by construction: the ACL travels with each chunk, and the security filter is applied **inside the search query** on every call, so unauthorised content is never retrieved in the first place. Because the whole platform is defined as code and proven in non-production — including an explicit **negative test** that a user is *denied* content they can't see — you validate the security boundary before any real content or real users are involved.

How it works

1. **Infrastructure as code** — a **Bicep** template (`AIRAG-infra.bicep`) is run **what-if first**, then deploys the platform: an **Azure AI Search** service, an **Azure OpenAI** resource with an **embedding** deployment and a **chat** deployment, a **storage account** for source documents, a **user-assigned managed identity** for keyless service-to-service auth, and (per the category requirements) **private endpoints** with public network access disabled.

2. **Index creation** — the pipeline creates the index with the fields RAG needs: `id`, `content` (chunk text), `contentVector` (the embedding, with a vector search + HNSW profile), source metadata (`title`, `url`, `lastModified`), and the **security field** `group_ids` (`Collection(Edm.String)`, filterable) — plus a **semantic configuration** so semantic ranking can run.

3. **Ingestion + embedding** — `AIRAG-Ingest.py` reads each source document **together with its permission set** (the Entra group object IDs that can read it), chunks the text, embeds each chunk, and **stamps every chunk with the parent document's** `group_ids`. Nothing is written to the index without an ACL.

4. **Permission-trimmed retrieval** — at query time the orchestration resolves the **caller's** Entra group memberships and executes the search with the filter `group_ids/any(g: search.in(g, '<caller group ids>'))`, combined with hybrid + semantic ranking. Only permitted, relevant chunks come back.

5. **Grounded generation** — the prompt flow (`RAG Flow – Orchestration – NonProd`) passes those chunks to the Azure OpenAI **chat** deployment inside a grounded prompt and returns the answer **with citations**; if no permitted chunk supports an answer, the flow returns "I don't have a source for that" rather than inventing one.

6. **Evaluation + monitoring** — an evaluation run (`RAG Eval – Groundedness – NonProd`) scores a test-question set for groundedness and relevance, and Application Insights captures latency, token usage, and errors for ongoing monitoring.

What you can control

• **Chunking** — chunk size and overlap (retrieval precision vs. context completeness).

• **Retrieval mix** — keyword-only, vector-only, or hybrid; whether **semantic ranking** is on; and `top-k` (how many chunks feed the model).

• **The security model** — which source-permission field maps to `group_ids`, and whether trimming is by **Entra group** (default) or another ACL scheme; trimming is **on by default and non-optional** in this design.

• **Grounding behaviour** — the system prompt, the "refuse if unsupported" instruction, and the citation format.

• **Models** — which embedding model and which chat model deployment are used, and their token/rate limits.

• **Evaluation thresholds** — the minimum groundedness/relevance scores that gate expansion.

Limitations (be honest with buyers)

• **Permission-trimming is only as correct as the ACLs you feed it.** The platform enforces the `group_ids` you stamp on each chunk; **you** must map each source system's real permissions to those group IDs accurately. A wrong or missing ACL at ingestion is a leak the search filter cannot catch. The validation script's **negative test** exists precisely to catch this before go-live.

• **Azure AI Search semantic ranking** and some **integrated vectorization** features have **tier requirements and, in places, preview status** — *verify current availability, tier, and pricing in your region and subscription.*

• **Azure OpenAI model and region availability, quotas (TPM), and pricing vary** and models are versioned/retired on a schedule — *verify the current models, versions, regions, and quota in your subscription.*

• RAG grounds and cites; it does **not** guarantee a correct answer if your source content is wrong, stale, or contradictory. Content quality and **data-classification** are upstream of this platform and are the compliance team's remit.

• **Acceptable-AI-use, content-safety thresholds, and records/regulatory** determinations are the customer's legal / trust-and-safety / compliance decisions (see `global-requirements-ai.md` G6/G7). This guide delivers the technical controls, not the policy.

Auth and connections used

• **Microsoft Entra ID** — the identity behind the **user-assigned managed identity** (keyless auth to Search and OpenAI) and the source of the **caller's group memberships** used for permission-trimming.

• **Azure Resource Manager** — the Bicep deployment operates against ARM with **Owner** or scoped **Contributor + role assignment** rights on the target resource group.

• **Azure AI Search data plane** — index create/load and query, authenticated by managed identity with least-privilege data-plane roles (**Search Index Data Contributor** to load, **Search Index Data Reader** to query).

• **Azure OpenAI data plane** — embeddings and chat completions, authenticated by managed identity (**Cognitive Services OpenAI User**).

• **Microsoft Graph** (read-only) — resolves a user's **group memberships** and, where used, a document's permission set for the ACL mapping.

The standard deployment model

> ***Assess prerequisites → deploy the infra (bicep what-if → apply, non-prod) → create the permission-trimmed index → run the ingestion/embedding pipeline (dry-run first) → wire the grounded prompt flow → validate (including the permission-trimming NEGATIVE test) → evaluate groundedness/relevance → expand to more content and production.*** *The repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, with a* ***what-if gate*** *before infra apply, a* ***dry-run gate*** *before index writes, and the* ***permission-trimming negative test as a hard gate*** *before expansion.*

2. Requirements Guide — Enterprise RAG Platform on Azure (AI Search + Private Data)

Verify **every** requirement before deploying. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites — the Azure subscription and AI-service access, Entra + managed identity + RBAC, private networking, IaC + Python tooling, CI/CD, data source + classification, content-safety posture, and cost controls — are in `global-requirements-ai.md` (G1–G8) and are **not** repeated here. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim.

2.1 Azure AI Search service at a tier that supports semantic ranking + vector

Description: An **Azure AI Search** service (`RAG Search – Platform – NonProd`) on a tier that supports **vector search** and **semantic ranking** — Basic or higher; semantic ranking is a billable add-on you enable on the service. **Why:** The whole retrieval layer (hybrid keyword+vector, semantic re-ranking, and the `group_ids` security filter) runs in this service; the free tier and older configurations do not support the full feature set. **Verify:** `portal.azure.com` → **Azure AI Search** → the service's **Semantic ranker** setting and tier; `AIRAG-Discovery.py` reports the service, tier, and semantic status. **Implement-if-missing:** Create the service (deployed by `AIRAG-infra.bicep`), pick a supported tier, and enable **Semantic ranker**. *Verify current tier requirements, availability, and pricing for vector + semantic ranking in your region.*

2.2 Azure OpenAI resource with an embedding deployment and a chat deployment

Description: An **Azure OpenAI** resource (`RAG OpenAI – Platform – NonProd`) with two model deployments: an **embedding** model deployment `rag-embeddings` (e.g. `text-embedding-3-large`) and a **chat** model deployment `rag-chat` (e.g. `gpt-4o`). **Why:** Embeddings turn each chunk and each query into vectors for vector search; the chat model generates the grounded, cited answer. The embedding model used at **ingestion** must match the one used at **query** (same model + dimensions). **Verify:** `portal.azure.com` → **Azure OpenAI** → **Model deployments** show both deployments; `AIRAG-Discovery.py` lists the resource and deployments. **Implement-if-missing:** Request Azure OpenAI access if gated; create both deployments (Bicep provisions the resource; you confirm model

availability). *Verify current model names, versions, dimensions, regions, and TPM quota in your subscription — models are versioned and retired on a schedule.*

2.3 A source data set with a known, resolvable permission (ACL) model

Description: The private content to ground on (e.g. SharePoint/OneDrive documents, a file share, a database export) **plus** a way to determine, per document, the **Entra group object IDs** that are permitted to read it. **Why:** Permission-trimming is only correct if each chunk is stamped with the *right* ACL. You must be able to map every source document to the group IDs that can read it; a missing or wrong ACL is a leak the search filter cannot catch. **Verify:** For a sample of documents you can state which Entra groups may read them; `AIRAG-Discovery.py` reports whether the sample source exposes a resolvable permission set. **Implement-if-missing:** Define the mapping from source permissions to `group_ids` (e.g. SharePoint site/library groups → Entra groups). Where a source has no usable ACL, decide a default group and **document it**. *Data-classification and "who should see what" determinations are the compliance team's decision (see `global-requirements-ai.md` G6).*

2.4 Entra groups for the pilot content and test users

Description: At least **two** Entra security groups representing distinct audiences for the pilot — e.g. `RAG Readers – Finance – Pilot` and `RAG Readers – General – Pilot` — and **two test users**: one who is a member of the restricted group and one who is **not**. **Why:** The permission-trimming **negative test** requires a user who *should* be denied restricted content. Without a genuinely-unauthorised test user you cannot prove trimming works. **Verify:** `entra.microsoft.com` → **Groups** → both groups exist with the expected members; capture each **Object ID**. `AIRAG-Discovery.py` echoes the IDs you supply. **Implement-if-missing:** Create the two groups, add the restricted content's readers to one, and keep the second test user out of it. Record both **object IDs** for ingestion and testing.

2.5 Managed identity with least-privilege data-plane roles

Description: A **user-assigned managed identity** (`RAG Identity – Platform – NonProd`) granted **Search Index Data Contributor** (to load), **Search Index Data Reader** (to query), and **Cognitive Services OpenAI User** (to embed + chat). **Why:** Keys leak; the pipeline and orchestration authenticate keylessly via managed identity + RBAC, least-privilege (see `global-requirements-ai.md` G2). Loading and querying are deliberately separate roles. **Verify:** The identity exists and holds exactly those roles at the Search service and OpenAI resource scope; `AIRAG-Validate.py` confirms it can query but the *query* principal cannot write. **Implement-if-missing:** Bicep creates the identity; assign the three data-plane roles at the narrowest scope. Do **not** grant Owner/Contributor to the runtime identity.

2.6 Python 3.x with the RAG SDK toolchain

Description: Python 3.10+ with `azure-search-documents`, `openai` (Azure OpenAI), `azure-identity`, `azure-storage-blob`, and `promptflow` installed in a virtual environment. **Why:** Ingestion, embedding, querying, permission-trimming validation, and the evaluation flow are delivered as Python using these SDKs (see `global-requirements-ai.md` G4). **Verify:** `python --version` ≥ 3.10 and `pip show azure-search-documents openai promptflow` resolve; `AIRAG-Validate.py` checks module presence. **Implement-if-missing:** `python -m venv .venv` then `pip install azure-search-documents openai azure-identity azure-storage-blob promptflow`. *Verify current SDK versions — the Azure AI SDKs move fast; pin them.*

2.7 Private networking for the AI services

Description: A VNet with the ability to create **private endpoints** and **private DNS** for Azure AI Search and Azure OpenAI, so **public network access can be disabled** on both. **Why:** Public endpoints are a data-exfiltration surface; enterprise RAG on private data is private-by-default (see `global-requirements-ai.md` G3). The pipeline runs from a host with a network path to the private endpoints. **Verify:** The VNet/subnets and Private DNS zones exist or can be created; after deploy, `publicNetworkAccess` is **Disabled** on both services; `AIRAG-Validate.py` reports public-access state. **Implement-if-missing:** Provision the VNet/subnets/Private DNS and the private endpoints (Bicep parameters), and run the pipeline from a host on that network (peering / VPN / jumpbox). See the AI landing-zone solution for the full pattern.

2.8 Cost controls, quotas, and a non-prod environment with rollback

Description: A **budget + alerts**, an **Azure OpenAI TPM quota** sized for ingestion (embedding a large corpus is token-heavy), a named **non-production** resource group to prove the platform first, and a documented rollback. **Why:** Embedding a whole corpus and running eval loops consume tokens and search units; unmanaged usage causes bill shock (see `global-requirements-ai.md` G8). Index writes and infra changes must be proven in non-prod with a back-out. **Verify:** A budget/alert exists, TPM quota is set for `rag-embeddings`, the non-prod resource group name is fixed, and a written rollback exists. **Implement-if-missing:** Set the budget, request/confirm TPM quota, define `rg-rag-platform-nonprod` (`RAG RG – Platform – NonProd`), and write the rollback (delete the non-prod resource group / drop-and-rebuild the index). *Spend limits are the customer's business decision.*

Checklist

- Azure AI Search service (tier + **Semantic ranker** enabled) — `AIRAG-Discovery.py`
- Azure OpenAI resource + `rag-embeddings` + `rag-chat` deployments
- Source data with a resolvable per-document Entra-group ACL model
- Two pilot Entra groups + an authorised **and** an unauthorised test user (object IDs captured)
- User-assigned managed identity with Search Data Contributor/Reader + OpenAI User (least privilege)
- Python 3.10+ with azure-search-documents / openai / azure-identity / promptflow — `AIRAG-Validate.py`
- VNet + private endpoints + Private DNS; public network access disabled on both services
- Budget/alerts, embedding TPM quota, non-prod resource group, and rollback documented

Global Requirements — AI (category-level)

These requirements apply to every solution in the AI category (building and operating custom AI on Azure — Azure OpenAI / AI Foundry, RAG, Azure ML / MLOps, AI landing zones, content safety). Published once at the category level so listings stay small. Confirm before deploying any AI solution. Each entry: Description / Why / Verify / Implement-if-missing.

Custom AI touches sensitive data and can emit harmful or ungrounded output. Deploy private-by-default (no public endpoints, keyless auth), ground and permission-trim on your own data, put safety guardrails in front of generation, and pilot in non-production before production — content-acceptability and AI-use-policy decisions belong to the customer's legal / trust-and-safety / compliance functions, not this guide.

G1. Azure subscription with AI service access and rights

Description: An Azure subscription with access to the required AI services (Azure OpenAI / AI Foundry, Azure AI Search, Azure Machine Learning, Azure AI Content Safety) and Contributor/Owner at the deployment scope. Why: These solutions create and configure real, billable Azure AI resources. Verify: `az account show` succeeds; the AI resource providers are registered and the region is available. *Azure OpenAI/model access and regions vary — verify current availability in your subscription.* Implement: Obtain the subscription/role; request access to the gated AI services where required; pick a supported region.

G2. Entra identity, managed identity, and RBAC

Description: Entra ID identities plus managed identities for keyless service-to-service auth. Why: Keys leak; AI services should authenticate via managed identity + RBAC (least privilege). Verify: Managed identities can be assigned and the needed data-plane roles exist (e.g. Cognitive Services OpenAI User, Search Index Data Reader). Implement: Enable managed identity on the compute/app; assign least-privilege data-plane roles; store any unavoidable secret in Key Vault.

G3. Private networking capability

Description: A VNet with the ability to create private endpoints and private DNS for the AI services. Why: Public endpoints are a data-exfiltration surface; enterprise AI is private-by-default. Verify: The VNet/subnets and Private DNS zones exist or can be created; public access can be disabled. Implement: Provision the VNet, subnets, and Private DNS; plan private endpoints per service (see the AI landing-zone solution).

G4. IaC + Python tooling and source control

Description: Terraform/Bicep, the Azure CLI, Python 3.x with the relevant Azure AI SDKs, and Git. Why: AI platforms are delivered as versioned code (infra + pipelines + app), so they are reproducible and reviewable. Verify: `az version`, `terraform version`/`az bicep version`, `python --version`, and `git --version` return supported builds. *Verify current SDK/provider versions — the AI SDKs move fast.* Implement: Install and pin the toolchain; store all config in Git.

G5. CI/CD platform

Description: A pipeline (GitHub Actions / Azure DevOps) with OIDC/workload-identity federation to Azure. Why: Infra, model, and app deployments must run through an auditable, approval-gated pipeline — not a laptop. Verify: The pipeline can authenticate to Azure via OIDC and run a plan/deploy step. Implement: Configure the service connection with workload-identity federation; scope its rights.

G6. Data source and classification (for data-grounded solutions)

Description: The private data to ground on (documents, databases) with a known permission/ACL model, and a sensitivity-classification position. Why: RAG must respect source permissions (permission-trimming) and must not expose sensitive data through AI. Verify: The data source, its access model, and applicable sensitivity labels are known. Run the solution's discovery script. Implement: Inventory the data and its ACLs; align sensitivity labeling (see the Purview solutions). *Data-classification determinations are the compliance team's decision.*

G7. Content-safety and responsible-AI posture

Description: An organizational position on acceptable AI content, safety thresholds, and responsible-AI review. Why: Generative AI can emit harmful/ungrounded output and be jailbroken; the organization must define guardrails and review. Verify: A responsible-AI / acceptable-use policy exists; content-safety is enabled where generation occurs. Implement: Adopt an RAI/acceptable-use policy and enable content safety (see the content-safety solution). *Acceptable-content and RAI determinations are the customer's legal / trust-and-safety decision.*

G8. Cost controls and quotas

Description: Budgets, alerts, and per-service quotas (e.g. Azure OpenAI TPM) appropriate to the workload. Why: AI services bill on tokens/compute; unmanaged usage causes bill shock. Verify: Budgets/alerts exist and quotas are set for the deployed services. Implement: Set budgets, cost alerts, and quotas/rate limits before broad rollout. *Spend limits are the customer's business decision.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Enterprise RAG Platform on Azure (AI Search + Private Data)

Follow these steps **in order**. Infrastructure is deployed by **Bicep** (`az deployment group what-if` **before** `create`), then the index, ingestion, orchestration, and evaluation are built with **Python**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Everything runs* ***what-if / dry-run first*** *and on the* ***non-production*** *resource group first. Deploy the infra with* `what-if`*, build the index, ingest a* ***small pilot set in dry-run*** *before writing, then run the* ***permission-trimming negative test*** *— expanding only after it passes. Keep a rollback (delete the non-prod resource group / drop-and-rebuild the index) for every change.*

Naming (set once, reuse verbatim)

- Non-prod resource group: `RAG RG – Platform – NonProd` (`rg-rag-platform-nonprod`)
- Azure AI Search service: `RAG Search – Platform – NonProd` (`srch-rag-nonprod`)
- Search index: `RAG Index – Content – NonProd` (`rag-index-nonprod`)
- Azure OpenAI resource: `RAG OpenAI – Platform – NonProd` (`oai-rag-nonprod`)

- Embedding deployment: `RAG Embeddings – Model – NonProd` (`rag-embeddings`, e.g. `text-embedding-3-large`)
- Chat deployment: `RAG Chat – Model – NonProd` (`rag-chat`, e.g. `gpt-4o`)
- Source storage account: `RAG Storage – Source – NonProd` (`stragrsrcnonprod`)
- Managed identity: `RAG Identity – Platform – NonProd` (`id-rag-platform-nonprod`)
- Orchestration prompt flow: `RAG Flow – Orchestration – NonProd`
- Evaluation run: `RAG Eval – Groundedness – NonProd`
- Pilot reader groups: `RAG Readers – Finance – Pilot` (restricted), `RAG Readers – General – Pilot`
- Security (ACL) field on every document/chunk: `group_ids` (`Collection(Edm.String)`, filterable)

Step 1 — Assess and set variables

1. Run `AIRAG-Discovery.py` to capture the pre-change picture: subscription context, any existing Search/OpenAI resources, the pilot Entra group object IDs you supply, and whether the sample source exposes a resolvable permission set.
2. Copy `AIRAG-infra.bicepparam` (or a parameters file you create) and set: resource group `rg-rag-platform-nonprod`, region, Search tier, the OpenAI model names for `rag-embeddings` / `rag-chat`, the VNet/subnet + Private DNS IDs for the private endpoints, and `publicNetworkAccess = Disabled`.
3. Confirm the operator/pipeline host has a **network path to the private endpoints** — the later ingestion and query steps depend on it.

Step 2 — Deploy the infrastructure (Bicep what-if = the gate)

1. `az login`, then `az group create -n rg-rag-platform-nonprod -l <region>`.
2. Run **what-if** and read the whole plan — **nothing is created yet**:

```
az deployment group what-if -g rg-rag-platform-nonprod -f AIRAG-infra.bicep -p AIRAG-infra.bicepparam
```

1. Review every resource: the Search service (with **Semantic ranker**), the OpenAI resource + the two model deployments, the source storage account, the **user-assigned managed identity**, and the **private endpoints** with public access disabled.
2. Only when the plan matches intent and the change is approved:

```
az deployment group create -g rg-rag-platform-nonprod -f AIRAG-infra.bicep -p AIRAG-infra.bicepparam
```

1. Assign the runtime identity its **least-privilege data-plane roles** (Search Index Data Contributor + Reader on the Search service; Cognitive Services OpenAI User on the OpenAI resource) at the narrowest scope.

Step 3 — Create the permission-trimmed index

1. Run `AIRAG-Ingest.py --create-index-only` (no data written yet). This creates `rag-index-nonprod` with the fields RAG needs:

• `id` (key), `content` (chunk text, searchable), `title` / `url` / `lastModified` (metadata),

• `contentVector` (`Collection(Edm.Single)`, the embedding, tied to a **vector search profile + HNSW** config with the **same dimensions** as `rag-embeddings`),

• `group_ids` (`Collection(Edm.String)`, **filterable**) — the per-chunk ACL, and

• a **semantic configuration** naming `content` as the semantic content field so semantic ranking can run.

1. Confirm the index exists: `AIRAG-Validate.py --check-index` reports the field set and confirms `group_ids` is present and filterable. **An index without the `group_ids` field must never be loaded** — the validator fails the run if it is missing.

Step 4 — Ingest a small pilot set (dry-run first)

1. Run `AIRAG-Ingest.py --source <pilot path/site> --dry-run`. In dry-run the script **reads, chunks, and resolves each document's `group_ids`, and prints a per-document summary — but writes nothing.** Confirm every document resolved a **non-empty** `group_ids` (any document with no ACL is flagged; decide its group before proceeding).

2. Re-run **without** `--dry-run` to embed and load: `AIRAG-Ingest.py --source <pilot path/site>`. Each chunk is embedded with `rag-embeddings` and written with its parent document's `group_ids` stamped on it.

3. Deliberately include, in the pilot set, **at least one document readable only by** `RAG Readers – Finance – Pilot` — this is the document the negative test will try (and must fail) to retrieve as the unauthorised user.

Step 5 — Verify hybrid + semantic retrieval works

1. Run `AIRAG-Validate.py --query "<a question answered by the pilot content>" --as-group <RAG Readers – Finance – Pilot object id>`.

2. Confirm the query returns relevant chunks, that **hybrid** (keyword + vector) and **semantic ranking** are both applied (the script prints the search mode and the semantic reranker score), and that citations point back to the correct source documents.

Step 6 — Run the PERMISSION-TRIMMING negative test (the hard gate)

1. Run the **same query** as the **unauthorised** user's groups — omit the Finance group:

```
AIRAG-Validate.py --query "<question that only the Finance-only document answers>" --as-group <RAG Readers – General – Pilot object id>
```

1. **Expected — the gate:** the restricted document's chunks are **NOT** returned (the security filter `group_ids/any(g: search.in(g, '<caller groups>'))` excludes them), and the grounded flow answers **"I don't have a source for that"** rather than leaking the content. `AIRAG-Validate.py --permission-trimming-test` runs both the authorised and unauthorised query and **fails the run if the unauthorised query returns any restricted chunk.**

2. **Do not proceed past this gate until the negative test passes.** A failure here means an ACL was mis-mapped at ingestion (Step 4) — fix the `group_ids` mapping and re-ingest, do not weaken the filter.

Step 7 — Wire the grounded orchestration (prompt flow)

1. Open the prompt flow `RAG Flow – Orchestration – NonProd`. Its nodes: **resolve caller groups** (Graph) → **embed the question** (`rag-embeddings`) → **hybrid+semantic search with the `group_ids` security filter** → **build the grounded prompt** → **chat completion** (`rag-chat`) → **format answer + citations**.

2. Set the grounded system prompt to instruct the model to **answer only from the supplied sources, cite each source, and explicitly refuse when the sources don't contain the answer.**

3. Run the flow interactively with the authorised and unauthorised test users and confirm the answer, the citations, and the refusal behaviour match Step 5/6.

Step 8 — Evaluate groundedness and relevance

1. Prepare a small **evaluation set** (questions with known good answers drawn from the pilot content).

2. Run the evaluation flow: `RAG Eval – Groundedness – NonProd` scores each answer for **groundedness** (supported by retrieved sources) and **relevance** (addresses the question). `AIRAG-Report.py` collects the scores.

3. Set your **pass thresholds** (e.g. groundedness ≥ your agreed bar) — these gate expansion. Investigate any low-groundedness answer (usually chunking too coarse, `top-k` too low, or a weak grounding prompt).

Step 9 — Turn on monitoring

1. Confirm the flow and services log to **Application Insights / Azure Monitor**: query latency, tokens per call, retrieval hit counts, and errors.

2. Run `AIRAG-Report.py` to produce the baseline posture: index size, **ACL coverage** (chunks with a non-empty `group_ids`), retrieval mode, eval scores, and public-access state of both services.

Step 10 — Expand to more content and production

1. Re-run `AIRAG-infra.bicep` targeting the **production** resource group (same template, prod parameters) — **what-if first** — to create the prod Search/OpenAI/identity/private endpoints.

2. Ingest the broader corpus in **batches**, each with a **dry-run ACL check first**; re-run the **permission-trimming negative test** after each new content domain is added (new content = new ACLs = re-prove trimming).

3. Re-validate with `AIRAG-Validate.py` and track index growth, ACL coverage, eval scores, and drift with `AIRAG-Report.py` on a schedule.

Order of operations

Assess → bicep what-if (gate) → deploy infra (non-prod) → create permission-trimmed index → ingest pilot (dry-run gate → load) → verify retrieval → PERMISSION-TRIMMING negative test (hard gate) → wire grounded flow → evaluate → monitor → expand to prod (what-if + re-test trimming per domain)

5. Testing Guide — Enterprise RAG Platform on Azure (AI Search + Private Data)

Run after the pilot ingest (Implementation Step 6), again after wiring the flow (Step 7–8), and after each expansion (Step 10). Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names, and the scripts are `AIRAG-Discovery.py`, `AIRAG-Ingest.py`, `AIRAG-Validate.py`, `AIRAG-Report.py`. **Test 4 is the headline gate — the platform does not ship until the permission-trimming negative test passes.**

Test 1 — Infra is private and keyless

Do: `AIRAG-Validate.py --check-infra`. **Expected:** `publicNetworkAccess` is **Disabled** on both `RAG Search – Platform – NonProd` and `RAG OpenAI – Platform – NonProd`; the runtime **managed identity** (not a key) authenticates; a query from *off* the private network fails (this is the point).

Test 2 — The index has the security field

Do: `AIRAG-Validate.py --check-index`. **Expected:** `rag-index-nonprod` exists with `content`, `contentVector` (vector profile + HNSW, dimensions matching `rag-embeddings`), the metadata fields, a **semantic configuration**, and the `group_ids` field present and **filterable**. If `group_ids` is missing, the check **fails** — an index without it can leak.

Test 3 — Every chunk carries an ACL

Do: `AIRAG-Report.py --acl-coverage`. **Expected: 100% ACL coverage** — every document/chunk in the index has a **non-empty** `group_ids`. Any chunk with an empty `group_ids` is reported as a defect to fix (re-ingest with the correct mapping) before go-live.

Test 4 — PERMISSION-TRIMMING negative test (the hard gate)

Do: `AIRAG-Validate.py --permission-trimming-test` — it runs the **same** question twice: once as a member of `RAG Readers – Finance – Pilot` (authorised for the restricted document) and once as `RAG Readers – General – Pilot` (**not** authorised). **Expected:**

- **Authorised query:** returns the restricted document's chunks and a grounded, cited answer.
- **Unauthorised query:** returns **zero** restricted chunks; the grounded flow answers **"I don't have a source for that."**

The script **fails the run** if the unauthorised query returns any restricted chunk. **This is the pass/fail gate for the whole solution.**

Test 5 — Hybrid + semantic ranking are actually applied

Do: `AIRAG-Validate.py --query "<pilot question>" --as-group <Finance object id> --explain`. **Expected:** The output shows the search executed **hybrid** (keyword + vector) with **semantic ranking** on, and lists a **semantic reranker score** per result — not a plain keyword search.

Test 6 — Grounded answer with citations

Do: Ask a question the pilot content answers, through `RAG Flow – Orchestration – NonProd`. **Expected:** The answer is drawn from the retrieved chunks and includes **citations** back to the specific source documents; the cited passages actually support the answer.

Test 7 — Refusal when unsupported

Do: Ask a question the pilot content does **not** answer. **Expected:** The flow answers **"I don't have a source for that"** (or equivalent) rather than inventing content — grounding + refusal behaviour is working.

Test 8 — Groundedness / relevance evaluation

Do: Run `RAG Eval – Groundedness – NonProd` over the evaluation set; collect with `AIRAG-Report.py --eval`. **Expected:** Groundedness and relevance scores meet your agreed thresholds; any answer below the groundedness bar is flagged for investigation (chunking, `top-k`, or grounding prompt).

Test 9 — Posture report

Do: Run `AIRAG-Report.py`. **Expected:** A clear baseline — index size, **ACL coverage (should be 100%)**, retrieval mode, eval scores, public-access state of both services, and token/latency summary from monitoring.

Pass/fail gate

Gate	Requirement
Non-prod pass	Tests 1–3, 5–7 pass; ACL coverage is 100%
Security gate (hard)	**Test 4 passes** — the unauthorised user is denied restricted content and no restricted chunk is returned
Quality gate	Test 8 meets your groundedness/relevance thresholds
Expand pass	Tests 1–4, 9 pass, and **Test 4 is re-run and passes for each new content domain** before it carries real users

6. Troubleshooting Guide — Enterprise RAG Platform on Azure (AI Search + Private Data)

Use `AIRAG-Discovery.py` for the pre-deploy picture, `AIRAG-Validate.py` for infra/index/retrieval and the permission-trimming test, and `AIRAG-Report.py` for posture, ACL coverage, and eval scores. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names.

A. The permission-trimming test FAILS (a restricted chunk reached an unauthorised user)

This is the most important failure to get right — do not ship until it passes.

Symptom	Cause	Remediation
Unauthorised query returns a restricted chunk	The chunk was stamped with the **wrong** `group_ids`, or the mapping defaulted to an "everyone" group	Fix the source→`group_ids` mapping in `AIRAG-Ingest.py`, **re-ingest** the affected content, re-run `AIRAG-Validate.py --permission-trimming-test`. **Never** widen the filter to make the test pass
Restricted chunk has empty `group_ids`	The document had no resolvable ACL and was loaded anyway	Empty ACLs must **block** ingestion, not default open — re-ingest with the corrected mapping; `AIRAG-Report.py --acl-coverage` must read 100%
Filter not applied at all	The query was built without the security filter, or filtered in app code after retrieval	Ensure the filter `group_ids/any(g: search.in(g, '<caller groups>'))` is passed **into the search call**; resolve the caller's groups from Graph, not from a static list
Caller groups resolved wrong	Graph group-membership lookup returned partial/no groups (e.g. transitive groups missing)	Use transitive membership (`memberOf` transitive) so nested-group readers are included; confirm the test user's expected groups with `AIRAG-Discovery.py`

B. Bicep deploy / infra failures

Symptom	Cause	Remediation
`what-if` / `create` fails with `AuthorizationFailed`	Operating account lacks rights to write resources or role assignments	Grant **Owner** or **Contributor + role-assignment** rights at the resource-group scope; re-run `what-if` then `create`

Symptom	Cause	Remediation
Semantic ranker not available / not enabled	Service tier too low, or semantic ranker not turned on	Deploy a supported tier and enable **Semantic ranker** on the service. *Verify current tier requirements and pricing in your region*
Model deployment fails (`rag-embeddings` / `rag-chat`)	Model/version not available in the region, or no quota	Pick a currently-available model/version + region; request TPM quota. *Verify current model availability and quota in your subscription*
Private endpoint / DNS errors	Private DNS zone not linked, or subnet policy blocks the endpoint	Link the Private DNS zone to the VNet and confirm the subnet allows private endpoints; re-deploy

C. Retrieval returns nothing or irrelevant chunks

Symptom	Cause	Remediation
Authorised query returns 0 chunks	The security filter excludes everything (caller groups don't match any stamped `group_ids`), or the index is empty	Confirm the caller is in the expected group and that group's ID was stamped at ingest; confirm ingest ran (`AIRAG-Report.py`)
Results are keyword-ish, not semantic	Hybrid or semantic ranking not applied	Run `AIRAG-Validate.py --explain` — it prints the search mode and semantic score; enable semantic ranking and pass the vector + text query together
Vector search errors on dimensions	Embedding model dimensions ≠ the `contentVector` field dimensions, or query embedded with a different model than the chunks	Re-create the field / re-ingest so ingest and query use the **same** embedding model + dimensions
Good chunk exists but isn't retrieved	Chunking too coarse or `top-k` too low	Reduce chunk size / add overlap, raise `top-k`, re-ingest the sample and re-evaluate

D. Answers are wrong, uncited, or hallucinated

Symptom	Cause	Remediation
Answer not supported by sources	Weak grounding prompt, or `top-k` too low so the supporting chunk wasn't retrieved	Strengthen the "answer only from the sources / refuse if unsupported" instruction; raise `top-k`; re-run `RAG Eval – Groundedness – NonProd`

Symptom	Cause	Remediation
No citations	Flow not returning the source references	Ensure the flow maps each retrieved chunk's `title`/`url` into the citation output
Model answers when it shouldn't	Refusal instruction missing or overridden	Add/repair the explicit refuse-when-unsupported instruction; confirm with Testing Guide Test 7
Low groundedness across the board	Retrieval quality (Section C) — fix retrieval before prompt/model	Tune chunking/`top-k`/semantic ranking first, then re-evaluate

E. Auth / keyless issues

Symptom	Cause	Remediation
`401/403` from Search or OpenAI	Managed identity missing the data-plane role, or code still using a key	Assign **Search Index Data Contributor/Reader** and **Cognitive Services OpenAI User**; switch the client to `DefaultAzureCredential` / managed identity
Query principal can write to the index	Runtime identity over-privileged	Split roles — the **query** path should hold **Data Reader** only; loading uses **Data Contributor**
`Public network access is disabled` when running the pipeline	Running from off the private network (by design)	Run ingestion/queries from a host with a path to the private endpoints (peering / VPN / jumpbox)

F. Cost / throttling

- **429 / rate-limit during ingestion** — embedding a large corpus exceeds the `rag-embeddings` TPM quota. Batch the ingest, add backoff (`AIRAG-Ingest.py` retries with backoff), and/or raise quota. *Verify current quota and pricing.*
- **Bill higher than expected** — re-embedding unchanged documents on every run. Ingest incrementally (only changed docs); keep the budget/alerts from Requirements 2.8 in place.

G. Diagnostics quick reference

```
python AIRAG-Discovery.py                                    # readiness (read-only)
python AIRAG-Validate.py --check-infra --check-index         # private/keyless + security field
python AIRAG-Validate.py --permission-trimming-test          # the hard gate
python AIRAG-Report.py --acl-coverage --eval                 # 100% ACL coverage + eval scores
az deployment group what-if -g rg-rag-platform-nonprod -f AIRAG-infra.bicep -p AIRAG-infra.bicepparam
```

H. When to escalate

If the infra reports private + keyless, the index has a filterable `group_ids` field, ACL coverage is 100%, the caller's groups resolve correctly (transitive), and the **permission-trimming test still fails** — stop, treat it as a security incident, keep the platform out of production, collect `AIRAG-Validate.py --permission-trimming-test` and `AIRAG-Report.py` output, and open a Microsoft support case with the index definition and a redacted query trace. Never move to production with a failing negative test.

7. Best Practices — Enterprise RAG Platform on Azure (AI Search + Private Data)

• **Trim on every query — never make it optional.** The `group_ids/any(g: search.in(g, '<caller groups>'))` security filter must be applied **inside the search call on every request**, resolved from the *caller's* Entra groups — not filtered after the fact in application code, and never bypassed for "admin" convenience. Post-filtering means the unauthorised content was still retrieved and could still leak through logs or a code bug. Filter at the index, always.

• **The ACL is only as good as the mapping — prove it with the negative test.** The search filter faithfully enforces whatever `group_ids` you stamped at ingestion; it cannot fix a wrong or missing ACL. Treat the **permission-trimming negative test** (a user *denied* content they can't see) as the hard gate, and **re-run it every time you onboard a new content domain** — new content means new ACLs to prove.

• **No document enters the index without a non-empty** `group_ids`**.** Fail the ingestion for any chunk that resolves an empty ACL rather than defaulting it to "everyone". `AIRAG-Report.py --acl-coverage` should read **100%** before go-live and stay there.

• **Keep it private and keyless.** Disable public network access on both the Search and OpenAI services, reach them over **private endpoints**, and authenticate with a **user-assigned managed identity** holding least-privilege data-plane roles (separate **Data Contributor** to load from **Data Reader** to query). No account keys in code or config.

• **Ground hard and let it refuse.** Instruct the model to **answer only from the retrieved sources, cite each one, and say "I don't have a source for that" when the sources don't support an answer.** A confident, uncited answer is a hallucination waiting to be quoted as fact.

• **Match the embedding model at ingest and query, and pin dimensions.** The vector field's dimensions must equal the `rag-embeddings` model output, and the **same** embedding model/version must be used to embed both chunks and queries — mixing them silently wrecks retrieval quality.

• **Tune chunking before you blame the model.** Most "the answer was wrong/incomplete" problems are retrieval problems — chunks too large (diluted matches) or too small (lost context), or `top-k` too low. Adjust chunk size/overlap and `top-k` and re-evaluate before touching the prompt or the model.

• **Name everything and reuse the name.** Use `<Purpose> – <Scope> – <Stage>` for the resource group, Search service, index, OpenAI resource, model deployments, identity, flow, and eval run, and reuse it verbatim across Bicep, Python, tests, and rollback so the estate stays auditable.

• **Watch the meter.** Embedding a large corpus and running eval loops are token-heavy — keep the **budget/alerts** and **TPM quota** in place, ingest in batches, and re-embed only changed documents on refresh.

• **Non-prod, then production — what-if first, every time.** Prove the whole platform (including Test 4) in `RAG RG – Platform – NonProd`, run `bicep what-if` before every apply, and keep a rollback (delete the non-prod resource group / drop-and-rebuild the index) written down before you change anything.

Where to go next

• **Azure AI Content Safety & Responsible-AI Guardrails** — put prompt-shield / jailbreak and harmful-content filters in front of the generation step in this flow.

• **AI Landing Zone (Governance, Private Networking, Cost)** — the full private-networking, policy, and cost-guardrail foundation the Search and OpenAI services should sit inside.

• **Azure AI Foundry / OpenAI Secure Enterprise Deployment** — hardening and quota/model governance for the Azure OpenAI resource behind this platform.

• **Microsoft Purview data classification & sensitivity labels** — get the source-side classification and labels right so the ACLs you map into `group_ids` are trustworthy.

• Adjacent **AI** solutions in the catalog cover MLOps pipelines and monitoring that complement this platform.

8. Deliverables Summary — Enterprise RAG Platform on Azure (AI Search + Private Data)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box — grounded, cited answers that honor source permissions
1. Solution Overview	What the RAG platform is — chunking/embedding, hybrid + semantic + vector search, permission-trimming, grounded orchestration, evaluation; benefits, limitations
2. Requirements Guide	Every solution-specific prerequisite with Verify / Implement-if-missing
Global Requirements (AI)	Category-wide prerequisites (G1–G8), published once
3. Implementation Guide	Exact, ordered steps — Bicep what-if → deploy infra → build permission-trimmed index → ingest (dry-run) → wire grounded flow → negative test → evaluate → expand
5. Testing Guide	Validation procedures with pass/fail gates — including the permission-trimming negative test
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations and where to go next

Automation & scripts

File	What it does
`AIRAG-Discovery.py`	Read-only Python discovery — subscription context, existing Search/OpenAI resources, the pilot Entra group object IDs, and whether the sample source exposes a resolvable permission set; a readiness picture before you deploy. Makes no changes
`AIRAG-infra.bicep`	Bicep for the platform — Azure AI Search (with Semantic ranker), Azure OpenAI + `rag-embeddings` and `rag-chat` deployments, source storage, a user-assigned managed identity, and private endpoints with public access disabled. **Deploy what-if first** (`az deployment group what-if` before `create`)
`AIRAG-Ingest.py`	The ingestion/embedding pipeline — creates the permission-trimmed index (`--create-index-only`), then chunks each source document, resolves its Entra-group ACL, embeds each chunk with `rag-embeddings`, and loads it with `group_ids` stamped on. **Defaults to** `--dry-run` (reads/chunks/resolves ACLs and reports, writes nothing) until you confirm
`AIRAG-Validate.py`	Validation — checks private/keyless infra, the index security field, hybrid+semantic retrieval, and runs the `--permission-trimming-test` (authorised vs unauthorised query) that **fails if any restricted chunk reaches an unauthorised caller**
`AIRAG-Report.py`	Posture report — index size, **ACL coverage (target 100%)**, retrieval mode, groundedness/relevance eval scores, public-access state, and token/latency summary from monitoring

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- The **Bicep** template and four **Python** scripts (`azure-search-documents`, `openai`, `promptflow`) in a single ZIP — delivered as a **Complete Solution Package (PDF + Word + scripts)**.
- A repeatable process following the repeatable model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with a what-if gate before infra apply, a dry-run gate before index writes, and the permission-trimming negative test as a hard gate before expansion.

Outcome

AI that answers from your private data, shows its sources, and never returns content a user isn't allowed to see — deployed private-by-default, proven in non-production (including the permission-trimming negative test) before it carries real content or real users, and reusable on every content set you onboard.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AIRAG-Discovery.py

Purpose: Read-only discovery for the "Enterprise RAG Platform on Azure (AI Search + Private Data)" solution. Captures the pre-change picture BEFORE any Bicep apply or index write: the subscription in context, any existing Azure AI Search and Azure OpenAI resources (with public-network-access state), the pilot Entra group object IDs you supply (and whether a test user is a member), and whether a sample source document exposes a resolvable permission (ACL) set to map into `group_ids`. Makes NO changes.

How to run: python -m venv .venv && . .venv/bin/activate # (Windows: .venv\\Scripts\\activate) pip install azure-identity azure-mgmt-search azure-mgmt-cognitiveservices msgraph-core python AIRAG-Discovery.py --subscription <sub id> \ search-rg rg-rag-platform-nonprod \ finance-group <RAG Readers - Finance - Pilot object id> \ general-group <RAG Readers - General - Pilot object id> \ test-user <unauthorised test user UPN or object id> Auth: DefaultAzureCredential (az login, or a managed identity). Read-only roles are enough (Reader on the resource group; Directory Readers / Group.Read.All on Graph).

AIRAG-Ingest.py

Purpose: The ingestion/embedding pipeline for the "Enterprise RAG Platform on Azure" solution. Two jobs: 1. --create-index-only : create the Azure AI Search index `rag-index-nonprod` with the fields RAG needs — including the SECURITY field `group_ids` (Collection(Edm.String), filterable), a vector field `contentVector` tied to an HNSW/vector-search profile, and a semantic configuration for semantic ranking. 2. (ingest) : for each source document, resolve the Entra group object IDs permitted to read it, chunk the text, embed each chunk with the `rag-embeddings` model, and load each chunk into the index with the parent document's `group_ids` STAMPED on it. No chunk is ever written without a non-empty ACL — this is what makes permission-trimming enforceable at query time.

How to run: pip install azure-search-documents azure-identity openai azure-storage-blob 1) create the index (safe; creates schema only) python AIRAG-Ingest.py --create-index-only \ search-endpoint https://srch-ragnonprod.search.windows.net \ index rag-index-nonprod 2) dry-run the pilot content (writes nothing — inspect ACL resolution) python AIRAG-Ingest.py --source ./pilot-docs --index rag-index-nonprod \ search-endpoint https://srch-ragnonprod.search.windows.net \ openai-endpoint https://oai-ragnonprod.openai.azure.com 3) commit (embed + load) once dry-run looks correct python AIRAG-Ingest.py --source ./pilot-docs --index rag-index-nonprod ... --commit

AIRAG-Report.py

Purpose: Read-only reporting for the "Enterprise RAG Platform on Azure" solution. Produces the baseline posture picture: index size (document/chunk count), ACL COVERAGE (the share of chunks carrying a non-empty `group_ids` — the target is 100%), the retrieval mode in use, groundedness/relevance evaluation scores (from the prompt-flow eval run `RAG Eval - Groundedness - NonProd`), the public-network-access state of the Search and OpenAI services, and a token/latency summary from monitoring. Makes NO changes.

How to run: pip install azure-search-documents azure-identity python AIRAG-Report.py --acl-coverage --eval \ search-endpoint https://srch-ragnonprod.search.windows.net --index rag-index-nonprod Optional inputs: --eval-results <promptflow eval output json> to summarise groundedness and relevance. Auth: DefaultAzureCredential (Search Index Data Reader is enough).

AIRAG-Validate.py

Purpose: Validation for the "Enterprise RAG Platform on Azure" solution. Confirms the platform is private + keyless, the index carries the mandatory security field, retrieval uses hybrid + semantic ranking, and — the headline check — runs the PERMISSION-TRIMMING NEGATIVE TEST: it issues the SAME query as an authorised caller and as an UNAUTHORISED caller and FAILS the run if any restricted chunk reaches the unauthorised caller. The security filter under test is: group_ids/any(g: search.in(g, '<caller group ids>')) applied INSIDE the search call on every request — never as an after-the-fact filter.

How to run: pip install azure-search-documents azure-identity azure-mgmt-search openai python AIRAG-Validate.py --check-infra --check-index \ search-endpoint https://srch-ragnonprod.search.windows.net --index rag-index-nonprod python AIRAG-Validate.py --query "<pilot question>" --as-group <Finance obj id> --explain \ search-endpoint https://srch-ragnonprod.search.windows.net --index rag-index-nonprod \ openai-endpoint https://oai-ragnonprod.openai.azure.com THE HARD GATE: python AIRAG-Validate.py --permission-trimming-test \ restricted-query "<question only the Finance-only doc answers>" \ auth-group <Finance obj id> --unauth-group <General obj id> \ search-endpoint ... --index rag-index-nonprod --openai-endpoint ...

AIRAG-infra.bicep

Purpose: // Deploy the RAG platform's Azure infrastructure as code, private-by-default: // - Azure AI Search (Semantic ranker enabled; hosts the permission-trimmed index) // - Azure OpenAI + an embedding deployment (rag-embeddings) and a chat // deployment (rag-chat) // - Storage account for source documents (RAG Storage - Source - NonProd) // - User-assigned managed identity (keyless service-to-service auth) // - Public network access DISABLED on Search + OpenAI (reach them via private // endpoints created in your networking stack / the AI landing zone) // // HOW TO RUN (ALWAYS what-if first — never blind-apply; non-prod resource group first) // az login // az group create -n rg-rag-platform-nonprod -l <region> // az deployment group what-if -g rg-rag-platform-nonprod \ // -f AIRAG-infra.bicep -p AIRAG-infra.bicepparam // <-- review (the gate) // az deployment group create -g rg-rag-platform-nonprod \ // -f AIRAG-infra.bicep -p AIRAG-infra.bicepparam // only after review+approval // Rollback: az group delete -n rg-rag-platform-nonprod // destroys THIS non-prod stack // // After deploy, assign the managed identity LEAST-PRIVILEGE data-plane roles: // - Search Index Data Contributor + Search Index Data Reader (on the Search service) // - Cognitive Services OpenAI User (on the OpenAI resource) // (role assignments are intentionally kept OUT of this template — grant at the // narrowest scope with your governance process.) // // EXPECTED RESULTS // A private, keyless RAG foundation — Search (semantic) + OpenAI (embeddings + chat) + // source storage + managed identity, public access disabled — ready for the Python // ingestion pipeline (AIRAG-Ingest.py) to build the permission-trimmed index. // // NOTES // White-label. What-if first. Non-prod first. Model names/versions, Search tier // feature/pricing, and semantic-ranker availability move between GA and preview and // vary by region — values here are placeholders you MUST verify for your subscription. // ==========